UNIVERSITY OF ASIA PACIFIC

Department of Computer Science and Engineering

Course Code: CSE 212
Course Title: Database Systems Lab

Submitted by:

Team: EventCore
Name: Hamika Akter Rupa
Reg. No.: 23201146
Name: Shah Nusrat Unzom Prethela
Reg. No.: 23201144
Name: Sumaiya Islam
Reg. No.: 23201161

Semester : 2 nd Year 2 nd Semester
Section : C2

Submitted to:

Alif Ruslan
Lecturer, Dept. of CSE,
University of Asia Pacific

Table Of Content

1. Project Description 3
2. Entities (Tables) 4
3. Table of Work 6
4. Relationship Between Entities 10
- ❖ Customer— Event 10
- ❖ Package — Event 10
- ❖ Venue — Event 10
- ❖ Event — Payment 11
- ❖ Event — STAFF_ASSIGNMENT — Staff 11
- ❖ Management — STAFF_ASSIGNMENT 12
- ❖ Event — Event_History 12

5. ER- Diagram 14
6. Schema- Diagram 15
7. SQL Queries 16
- ❖ Reschedule Birthday events to 2025-11-05 and mark as 'Rescheduled 16
- ❖ List events with status 'Pending' or 'Rescheduled'. 16
- ❖ Update Customer table and show all records. 17
- ❖ List package names with booking count, ordered by count descending. 18
- ❖ List upcoming events with ID, type, date, customer, and package, ordered by date. 19
- ❖ Update payment for event ID 2 to 'Paid' with amount 50,000; show all payments. 19
- ❖ Get total payment revenue, average venue rent, and list events with status 'Confirmed'.20
- ❖ Update 21
- ❖ List events in next 30 days with ID, type, date, venue, customer, and package, ordered by date. 22
- ❖ List top 5 customers by total spending, showing name and amount, ordered highest first. 23
- ❖ List pending payments older than 10 days with event ID, customer, amount, and date.24
- ❖ Updating Staff Salaries Based on Role 24
- ❖ Total Revenue Generated by Each Package 24
- ❖ Add and Update Transaction Time in Payment Table 25
- ❖ Trigger to Prevent Double Booking of a Venue 26
- ❖ Average Rating of Events by Event Type 27

8. CEP Mapping 27
9. Conclusion 30

1. Project Description

This project presents a fully structured Event Management System that is designed to digitalize and automate the complete workflow of managing events. It provides an efficient way of recording, monitoring, and analyzing all the key operations involved in the event-management business, starting from customer registration to event execution, staff allocation, venue selection, payment tracking, and customer feedback.

The key goal of this project is to develop a relational database model that ensures accuracy, redundancy elimination, and smooth information flow within the interconnected entities: Customers, Venues, Packages, Staff, Events, and Payments. The system targets real-world operational needs, which include avoiding venue double-booking, revenue calculations, identification of the best customers, tracking event performance through reviews, and event history.

Features that assure reliability and scalability include:

- **Normalized database tables** for maintaining clean data.
- **Structured foreign-key** relationships that guarantee referential integrity.
- **Triggers** at the form level for maintaining business rules automatically, such as preventing double booking.
- **SQL queries for analysis to support decision-making**: most booked packages, highest-rated events, total revenue generated, among others.
- **Dynamic modification functionalities** that allow updating customer information, payments, and staff information.

Overall, the Event Management System provides a centralized source of information that lets an event organizer have fast access to correct and timely information on all events in progress or already finished. It enhances operational efficiency, boosts customer satisfaction, and offers data-driven insights supportive of long-term business growth.

This project demonstrates how database technology can be applied to solve real-life industry challenges through systematic design, relational mapping, and powerful SQL operations.

2. Entities (Tables)

I. Management

- manager_id
- name
- role
- email
- phone
- share_percent

II. Customer

- customer_id
- name
- email
- phone
- customer_city
- customer_address
- customer_type

III. Venue

- venue_id
- venue_name
- location
- capacity
- rent

IV. Package

- package_id
- package_name
- description
- price

V. Staff

- staff_id
- name
- email
- role
- role_description
- phone
- salary
- manager_id (FK)

VI. Event

- event_id
- customer_id (FK)
- venue_id (FK)
- package_id (FK)
- event_type
- event_date
- staff_id (FK)
- status
- duration_days

VII. Payment

- payment_id
- event_id (FK)
- amount
- payment_status
- payment_method
- transaction_id
- payment_date
- Transaction_time

VIII. Event_History

- history_id
- event_id (FK)
- event_type
- event_date
- status
- notes

IX. Review

- review_id
- event_id (FK)
- customer_id (FK)
- rating
- comments
- review_date

3. Table of Work

1. Management

a) manager_id (Primary Key)
b) name
c) function
d) Email
e) telephone
f) share_percent
Purpose:
This aspect of the system tracks the main management team of the company: who they are, their function, how to contact them, and what percent of the company they own. In other words, it is a record of who runs the organization from the top.

2. Customer

a) customer_id (Primary Key)

b) name (Not Null)
c) email (Unique)
d) telephone
e) customer_city
f) customer_address
g) customer_type (Regular/VIP)
Purpose:
This would be the entity containing all the required information about the customers: their basic information, location, and whether they are a regular customer or a VIP. Through this, the system identifies which customer booked what event and thus communicates appropriately.

3. Venue

a) venue_id (Primary Key)
b) venue_name
c) location
d) capacity
e) rent
Purpose:
This section holds information on all venues that the company can use for events. Details are included on the size of the venue, where it is located, and how much it will cost to rent. This helps customers pick a place to suit their needs and budget.

4. Package

a) package_id (Primary Key)
b) package_name
c) description
d) price
Purpose:
This entity enumerates the different packages of services the company provides. Here are the packages, like Silver, Gold, or Platinum, kept with short descriptions and prices. It helps the customer to learn what is included in each package before making a choice.

5. Staff

a) staff_id (Primary Key)
b) name
c) e-mail (Unique)
d) function
e) A description of the job
f) telephone
g) salary
h) manager_id (Foreign Key → Management.manager_id)
Purpose:
This entity would maintain the details of all employees working under the organization: what their job is, what responsibilities they handle, their salary, and which manager oversees them. This information is used by the system for assigning staff to various events when needed.

6. Event

a) event_id (Primary Key)
b) customer_id (Foreign Key → Customer.customer_id
c) venue_id (Foreign Key → Venue.venue_id)
d) package_id (Foreign Key → Package.package_id)
e) event_type
f) event_date
g) staff_id (Foreign Key → Staff.staff_id)
h) status (Confirmed, Pending, Rescheduled, Cancelled)
i) duration_days
Purpose:
It is the entity that holds complete information on each event booking. It integrates all the information about customers, venues, packages, and staff in one place. As it holds all the main information concerning an event, it works as the central part of the whole system.

7. Payment

a) payment_id (Primary Key)
b) event_id (Foreign Key → Event.event_id)

c) quantity
d) payment_status (Paid/Pending)
e) payment_method (Bkash, Card, Cash, Bank Transfer)
f) transaction_id
g) payment_date
h) transaction_time
Purpose:
This entity stores all the details related to every transaction associated with events. It includes the amount paid, the mode used, and whether the payment is processed or not. It helps in maintaining a clear financial record for every booked event.

8. Event_History

a) history_id (Primary Key)
b) event_id (Foreign Key → Event.event_id)
c) event_type
d) event_date
e) status
f) notes
Purpose: This tracks prior changes to an event. If an event was moved, or its status changed to cancelled, or even updated, the previous versions of that information are tracked here. It keeps a history of what transpired over time with regard to that event.

9. Review

a) review_id (Primary Key)
b) event_id (Foreign Key → Event.event_id)
c) customer_id (Foreign Key → Customer.customer_id)
d) rating
e) comments
f) review_date

Purpose: This is the entity that holds customer feedback on the event after its completion. It holds their ratings and comments, thus helping the company understand how good the event was and where improvements can be made next time.

4. Relationship Between Entities

1. Customer — Event

Relationship Type: One-to-Many (1:M)

One Customer can book multiple Events. Each Event belongs to one Customer.

Explanation:

- A customer may book several events (e.g., wedding, birthday, corporate event).
- However, every event record must be linked to exactly one customer who created or booked it.
- This relationship represents the booking responsibility.

Foreign Key: Event.cust_id → Customer.cust_id

2. Package — Event

Relationship Type: One-to-Many (1:M)

One package can be applied to many events, but an event uses only one package.

Explanation:

- Packages (e.g., Silver, Gold, Premium) are predefined service bundles.
- Multiple events can choose the same package.
- An event cannot have more than one package at the same time.

Foreign Key: Event.package_id → Package.package_id

3. Venue — Event

Relationship Type: One-to-Many (1:M)

One venue can host many different events across different dates.

Explanation:

- Venues such as halls, gardens, banquet spaces can be reused across multiple events.
- An event must be assigned to exactly one venue.
- No event can take place without a venue.

Foreign Key: Event.venue_id → Venue.venue_id

4. Event — Payment

Relationship Type: One-to-One (1:1)

Explanation:

- Each event generates a single payment record (total amount billed).
- A payment ALWAYS corresponds to one event.
- If an event is canceled, its payment record can be updated accordingly.

Foreign Key: Payment.event_id → Event.event_id

5. Event — Staff — Staff_Assignment (Bridge Table)

Relationship Type: Many-to-Many (M: N) (using Staff_Assignment table)

Explanation:

- An event may require multiple staff members
  (e.g., photographer, decorator, lighting team, caterers).
- A staff member can work in multiple events on different dates.
- This creates an M: N relationship.

To resolve this, the schema includes:

Staff_Assignment (event_id, staff_id, admin_id, assignment_date)

This acts as a **bridge table**, converting M: N into two 1:M relationships:

1. Event → Staff_Assignment (1:M)
2. Staff → Staff_Assignment (1:M)

Foreign Keys:

- Staff_Assignment.event_id → Event.event_id
- Staff_Assignment.staff_id → Staff.staff_id

6. Management — Staff_Assignment

Relationship Type: One-to-Many (1:M)

One Admin can assign many staff members to different events.

Explanation:

- An admin is responsible for organizing events.
- They assign staff to events using the Staff_Assignment table.
- The admin performing the assignment is recorded using admin_id.

Foreign Key: Staff_Assignment.admin_id → Admin.admin_id

7. Event — Event_History

Relationship Type: One-to-Many (1:M)

One event can have multiple history entries.

Explanation:

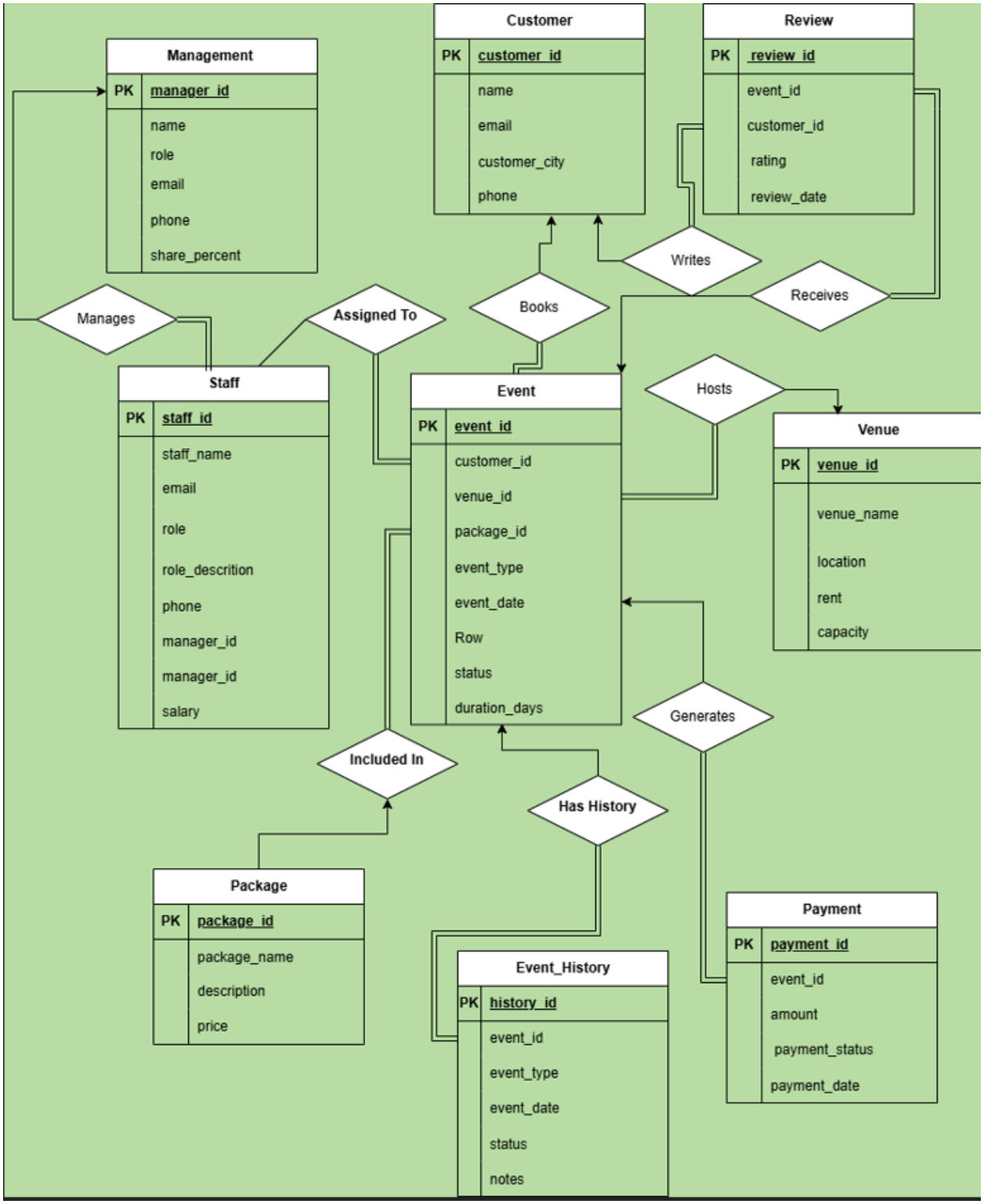
- Event history stores changes over time, such as:
  - Payment status updates
  - Date rescheduling
  - Venue changes
  - Admin actions
- Since an event may be updated several times, multiple history records are stored for a single event.

Foreign Key: Event_History.event_id → Event.event_id

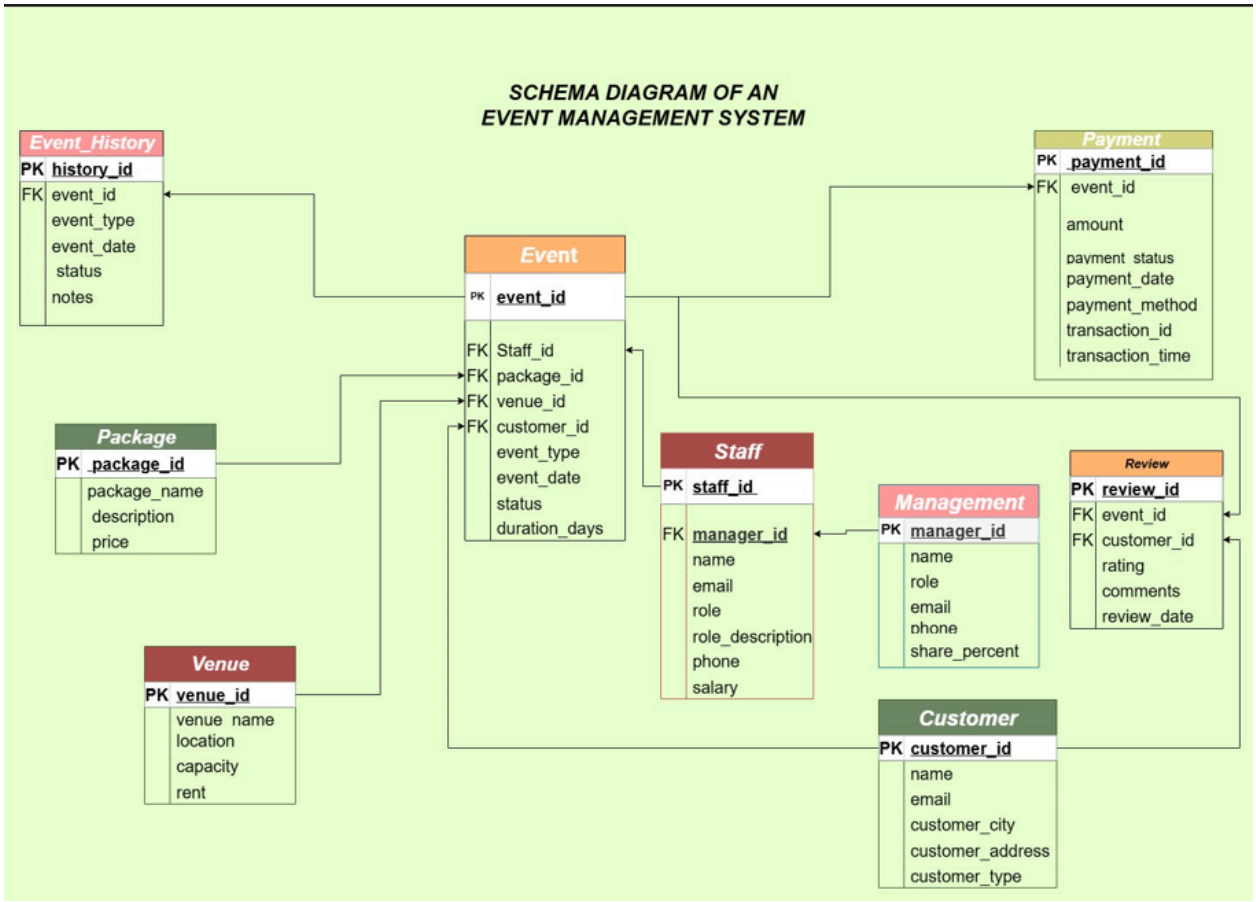
RELATIONSHIP SUMMARY TABLE

Entity A	Relationship	Entity B	Cardinality
Customer	books	Event	1 : M
Package	assigned to	Event	1 : M
Venue	hosts	Event	1 : M
Event	has	Payment	1 : 1
Event	assigned with	Staff	M : N (via Staff_Assignment)
Admin	manages	Staff_Assignment	1 : M
Event	generates	Event_History	1 : M

5. ER Diagram


Management
PK manager_id
name
role
email
phone
share_percent
Customer
PK customer_id
name
email
customer_city
phone
Review
PK review_id
event_id
customer_id
rating
review_date
Manages
Assigned To
Books
Writes
Receives
Staff
PK staff_id
staff_name
email
role
role_descrition
phone
manager_id
manager_id
salary
Event
PK event_id
customer_id
venue_id
package_id
event_type
event_date
Row
status
duration_days
Hosts
Venue
PK venue_id
venue_name
location
rent
capacity
Generates
Included In
Has History
Package
PK package_id
package_name
description
price
Payment
PK payment_id
event_id
amount
payment_status
payment_date
Event_History
PK history_id
event_id
event_type
event_date
status
notes

6. Schema Diagram



7.SQl Queries

1. **Find reschedule all Birthday events to the date 2025-11-05 and update their status to ‘Rescheduled’. After the update, display all records from the Event table.**

UPDATE Event
SET event_date = '2025-11-05', status = 'Rescheduled'
WHERE event_type = 'Birthday';
SELECT * FROM Event;

This SQL statement updates all records in the Event table where event_type is 'Birthday', changing their event_date to '2025-11-05' and status to 'Rescheduled'. The SELECT * FROM Event; then displays all events in the table, showing the updated values.

event_id	customer_id	venue_id	package_id	event_type	event_date	status	duration_days
1	1	1	3	Wedding	2025-10-15	Confirmed	3
2	2	2	2	Birthday	2025-11-05	Rescheduled	1
3	3	3	2	Conference	2025-12-10	Confirmed	2
4	4	1	1	Concert	2025-11-10	Rescheduled	1
5	5	4	3	Seminar	2025-11-20	Confirmed	1

2. **Find all events whose status is either ‘Pending’ or ‘Rescheduled’.**

SELECT * FROM Event WHERE status IN ('Pending','Rescheduled');
SELECT name, role, salary
FROM Staff
WHERE salary > (SELECT AVG(salary) FROM Staff);
The first query retrieves all events whose current status is either ‘Pending’ or ‘Rescheduled’.

The second query displays the name, role, and salary of staff members whose salary is higher than the average salary of all staff in the Staff table.

event_id	customer_id	venue_id	package_id	event_type	event_date	staff_id	status	duration_days
2	2	2	2	Birthday	2025-11-05	NULL	Rescheduled	1
4	4	1	1	Concert	2025-11-10	NULL	Rescheduled	1

3. **Update the Customer table and display all records from the Customer table.**

```
ALTER TABLE Customer
ADD customer_address VARCHAR(150) AFTER customer_city,
ADD customer_type ENUM('Regular','VIP') DEFAULT 'Regular' AFTER
customer_address;
UPDATE Customer
SET customer_address = 'House 12, Road 5, Gulshan, Dhaka', customer_type =
'VIP'
WHERE customer_id = 1;

UPDATE Customer
SET customer_address = 'House 44, Nasirabad, Chittagong', customer_type =
'Regular'
WHERE customer_id = 2;

UPDATE Customer
SET customer_address = 'Zindabazar, Sylhet', customer_type = 'Regular'
WHERE customer_id = 3;

UPDATE Customer
SET customer_address = 'Road 8, Dhanmondi, Dhaka', customer_type = 'Regular'
WHERE customer_id = 4;

UPDATE Customer
SET customer_address = 'Ranibazar, Rajshahi', customer_type = 'VIP'
WHERE customer_id = 5;
```

SELECT * FROM Customer;

This script first alters the Customer table by adding two new fields: customer_address and customer_type (with types Regular or VIP). Then, it updates the address and type for each customer based on their customer_id. Finally, the SELECT * FROM Customer; query displays all customer records with the newly added information.

customer_id	name	email	phone	customer_city	customer_address	customer_type
1	Ayra	ayra@gmail.com	01711111111	Dhaka	House 12, Road 5, Gulshan, Dhaka	VIP
2	Shayla	shayla@gmail.com	01822222222	Chittagong	House 44, Nasirabad, Chittagong	Regular
3	Mark	mark@gmail.com	01933333333	Sylhet	Zindabazar, Sylhet	Regular
4	Fiza	fiza@gmail.com	01644444444	Dhaka	Road 8, Dhanmondi, Dhaka	Regular
5	Tanzim	tan@gmail.com	01555555555	Rajshahi	Ranibazar, Rajshahi	VIP

4. **Find each package name along with the number of times it has been booked. The result should be ordered in descending order of the number of times booked.**

SELECT p.package_name, COUNT(e.event_id) AS Times_Booked
FROM Event e
JOIN Package p ON e.package_id = p.package_id
GROUP BY p.package_id
ORDER BY Times_Booked DESC;

This query finds how many times each package has been booked. It joins the Event and Package tables, counts the number of events for each package, groups them by package, and then sorts the result in descending order based on the number of bookings.

package_name	Times_Booked
Gold Package	2
Platinum Package	2
Silver Package	1

5. **List all upcoming events (events occurring today or in the future), showing the event ID, event type, event date, customer name, and package name. The results should be ordered by event date.**

```
SELECT e.event_id, e.event_type, e.event_date, c.name AS Customer, p.package_name

FROM Event e

JOIN Customer c ON e.customer_id = c.customer_id

JOIN Package p ON e.package_id = p.package_id

WHERE e.event_date >= CURDATE()

ORDER BY e.event_date;
```

This query retrieves all upcoming events (events on or after today), showing each event's ID, type, date, the customer's name, and the associated package. It joins the Event, Customer, and Package tables and orders the results by event date in ascending order.

event_id	event_type	event_date	Customer	package_name
2	Birthday	2025-11-05	Shayla	Gold Package
4	Concert	2025-11-10	Fiza	Silver Package
5	Seminar	2025-11-20	Tanzim	Platinum Package
3	Conference	2025-12-10	Mark	Gold Package

6. **Update the payment status to 'Paid' and the amount to 50,000 for the payment of the event with ID 2. After the update, display all records from the Payment table.**

```
UPDATE Payment
SET payment_status = 'Paid', amount = 50000
WHERE event_id = 2;
SELECT * FROM Payment;
```

This SQL command updates the Payment table by setting the payment_status to 'Paid' and amount to 50000 for the record where event_id is 2. The SELECT * FROM Payment; then retrieves all payment records, showing the updated values.

payment_id	event_id	amount	payment_status	payment_date
1	1	120000.00	Paid	2025-10-10
2	2	50000.00	Paid	2025-10-20
3	3	90000.00	Paid	2025-12-01
4	4	10000.00	Pending	2025-11-01
5	5	75000.00	Paid	2025-11-15

7. **Find the total revenue received from all payments. Find the average rent of all venues.Display all events that have the status 'Confirmed'.**

SELECT SUM(amount) AS Total_Revenue FROM Payment;

Total_Revenue
345000.00

SELECT AVG(rent) AS Average_Rent FROM Venue;

Average_Rent
98400.000000

SELECT * FROM Event WHERE status = 'Confirmed';

event_id	customer_id	venue_id	package_id	event_type	event_date	status	duration_days
1	1	1	3	Wedding	2025-10-15	Confirmed	3
3	3	3	2	Conference	2025-12-10	Confirmed	2
5	5	4	3	Seminar	2025-11-20	Confirmed	1

8. Update

```
ALTER TABLE Payment
ADD payment_method VARCHAR(50),
ADD transaction_id VARCHAR(50);
UPDATE Payment
SET payment_method = 'Bkash',
   transaction_id = 'TXN1001'
WHERE payment_id = 1;
UPDATE Payment
SET payment_method = 'Cash',
   transaction_id = 'TXN1002'
WHERE payment_id = 2;
UPDATE Payment
SET payment_method = 'Card',
   transaction_id = 'TXN1003'
WHERE payment_id = 3;
UPDATE Payment
SET payment_method = 'Bkash',
   transaction_id = 'TXN1004'
WHERE payment_id = 4;
UPDATE Payment
SET payment_method = 'Bank Transfer',
   transaction_id = 'TXN1005'
WHERE payment_id = 5;
SELECT * FROM Payment;
```

This SQL script first alters the Payment table by adding two new columns: payment_method (to store how the payment was made) and transaction_id (to store the transaction reference). Then, it updates each payment record with its corresponding

method and transaction ID. Finally, SELECT * FROM Payment; displays all payment details, including the newly added information.

payment_id	event_id	amount	payment_status	payment_date	payment_method	transaction_id
1	1	120000.00	Paid	2025-10-10	Bkash	TXN1001
2	2	50000.00	Paid	2025-10-20	Cash	TXN1002
3	3	90000.00	Paid	2025-12-01	Card	TXN1003
4	4	10000.00	Pending	2025-11-01	Bkash	TXN1004
5	5	75000.00	Paid	2025-11-15	Bank Transfer	TXN1005

9. List all events scheduled in the next 30 days, showing the event ID, event type, event date, venue name, customer name, and package name. The results should be ordered by event date.

```
SELECT e.event_id, e.event_type, e.event_date, v.venue_name, c.name AS
customer_name, p.package_name
FROM Event e
JOIN Venue v ON e.venue_id = v.venue_id
JOIN Customer c ON e.customer_id = c.customer_id
JOIN Package p ON e.package_id = p.package_id
WHERE e.event_date BETWEEN CURDATE() AND DATE_ADD(CURDATE(),
INTERVAL 30 DAY)
ORDER BY e.event_date;
```

This query retrieves upcoming events scheduled within the next 30 days, showing their event details along with the related venue, customer, and package information. It joins the Event, Venue, Customer, and Package tables, and then sorts the results by the event date in ascending order.

event_id	event_type	event_date	venue_name	customer_name	package_name
2	Birthday	2025-11-05	XinXian Convention Hall	Shayla	Gold Package
4	Concert	2025-11-10	Grand Palace	Fiza	Silver Package
5	Seminar	2025-11-20	Dream Hall	Tanzim	Platinum Package

10. Find the top 5 customers who have spent the most on events. Display the customer name along with the total amount they have spent, ordered from highest to lowest total.

This query finds the top 5 highest-paying customers. It joins the Customer, Event, and Payment tables, sums up the total payment made by each customer, groups the results by customer, and then orders them from the highest spender to the lowest, displaying only the first five customers.

```
SELECT c.name, SUM(pay.amount) AS Total_Spent
FROM Customer c
JOIN Event e ON c.customer_id = e.customer_id
JOIN Payment pay ON e.event_id = pay.event_id
GROUP BY c.customer_id
ORDER BY Total_Spent DESC
LIMIT 5;
```

name	Total_Spent
Ayra	120000.00
Mark	90000.00
Tanzim	75000.00
Shayla	50000.00
Fiza	10000.00

11. List all payments that are still pending and were made more than 10 days ago. Display the event ID, customer name, payment amount, and payment date.

```
SELECT e.event_id, c.name, pay.amount, pay.payment_date
FROM Payment pay
JOIN Event e ON e.event_id = pay.event_id
JOIN Customer c ON c.customer_id = e.customer_id
WHERE pay.payment_status = 'Pending' AND DATEDIFF(CURDATE(),
pay.payment_date) > 10;
```

This query retrieves all pending payments that are more than 10 days overdue. It joins the Payment, Event, and Customer tables to display the event ID, customer name, payment amount, and payment date for each such overdue payment.

12.Updating Staff Salaries Based on Role

```
UPDATE Staff
SET salary = salary * 1.10
WHERE role IN ('Event Manager',
'Coordinator');
SELECT * FROM Staff;
```

This SQL statement increases the salary by 10% for all staff members whose role is either 'Event Manager' or 'Coordinator'. The SELECT * FROM Staff; then displays all staff records,showing the updated salaries.

staff_id	name	email	role	role_description	phone	salary
1	Arif	arif@eventpro.com	Event Manager	Oversees all events and staff	01711110000	44000.00
2	Nadia	nadia@eventpro.com	Coordinator	Coordinates event logistics	01822223333	38500.00
3	Rahim	rahim@eventpro.com	Sound Engineer	Manages sound and audio setup	01933334444	30000.00
4	Sabbir	sabbir@eventpro.com	Lighting Technician	Handles lighting and stage setup	01644445555	28000.00
5	Farzana	farzana@eventpro.com	Catering Supervisor	Oversees catering and meals	01555556666	32000.00

13. Total Revenue Generated by Each Package

```
SELECT p.package_name,
SUM(pay.amount) AS Total_Revenue
FROM Event e
```

```
JOIN Package p ON e.package_id =
p.package_id
JOIN Payment pay ON e.event_id =
pay.event_id
GROUP BY p.package_id
ORDER BY Total_Revenue DESC;
```

This query calculates which event packages generated the most revenue. It joins Event, Package, and Payment tables, then groups the results by each package and sums up all payments to find the total revenue per package. Finally, it orders the packages from highest to lowest revenue.

package_name	Total_Revenue
Platinum Package	195000.00
Gold Package	140000.00
Silver Package	10000.00

14. Add and Update Transaction Time in Payment Table

```
ALTER TABLE Payment
ADD transaction_time TIME;
UPDATE Payment
SET transaction_time = '14:30:00'
WHERE payment_id = 1;
UPDATE Payment
SET transaction_time = '16:00:00'
WHERE payment_id = 2;
UPDATE Payment
SET transaction_time = '11:00:00'
WHERE payment_id = 3;
UPDATE Payment
SET transaction_time = '15:45:00'
WHERE payment_id = 4;
UPDATE Payment
SET transaction_time = '10:15:00'
```

This SQL script adds a new column transaction_time to store the exact time of each payment. Then, it updates the time for each payment record based on payment_id.

Finally, SELECT * FROM Payment; displays all payment records along with the newly added transaction time.

payment_id	event_id	amount	payment_status	payment_date	payment_method	transaction_id	transaction_time
1	1	120000.00	Paid	2025-10-10	Bkash	TXN1001	14:30:00
2	2	50000.00	Paid	2025-10-20	Cash	TXN1002	16:00:00
3	3	90000.00	Paid	2025-12-01	Card	TXN1003	11:00:00
4	4	10000.00	Pending	2025-11-01	Bkash	TXN1004	15:45:00
5	5	75000.00	Paid	2025-11-15	Bank Transfer	TXN1005	10:15:00

15. Trigger to Prevent Double Booking of a Venue

```
DELIMITER //
CREATE TRIGGER prevent_double_booking
BEFORE INSERT ON Event
FOR EACH ROW
BEGIN
DECLARE overlap_count INT;
SELECT COUNT(*) INTO overlap_count
FROM Event
WHERE venue_id = NEW.venue_id
AND (
(NEW.event_date BETWEEN event_date AND DATE_ADD(event_date, INTERVAL
duration_days - 1
DAY))
OR
(event_date BETWEEN NEW.event_date AND DATE_ADD(NEW.event_date,
INTERVAL
NEW.duration_days - 1 DAY))
);
IF overlap_count > 0 THE
SIGNAL SQLSTATE '45000'
SET MESSAGE_TEXT = 'Error: Venue already booked for the selected date range!'
END IF;
END;
```

The trigger ensures that a venue cannot be double-booked for overlapping events. In

other words, if a new event is being inserted into the Event table, the trigger checks whether the venue is already booked for any date that overlaps with the new event.

16. Average Rating of Events by Event Type

```
SELECT e.event_type, AVG(r.rating) AS
Avg_Rating
FROM Event e
JOIN Review r ON e.event_id = r.event_id
GROUP BY e.event_id
ORDER BY Avg_Rating DESC;
```

This query calculates the average rating for each event. It joins the Event and Review tables, groups the results by event, computes the average of all ratings per event, and orders the events from highest to lowest average rating.

event_type	Avg_Rating
Wedding	5.0000
Seminar	5.0000
Conference	4.0000

8. CEP Mapping

The CEP, or Celebration Event Planner, is a database system developed by a group to manage the complete workflow of an event management company. Our team designed a structured MySQL database that managed customers, venues, staff, management members, event packages, bookings, payments, reviews, and event history.

We designed all the tables with appropriate relationships so that data will always remain consistent and linked properly within the system. The project incorporates realistic features such as event scheduling, payment tracking, customer feedback, and a trigger for the prevention of double-booking of venues.This project demonstrates how an event management system can be organized and operated efficiently by a well-planned relational database created by our group.

K's (Knowledge) Mapping for CEP Project

K's	Attributes	How K's are addressed through our project	CO	PO
K3	Engineering Fundamentals	The project uses core engineering fundamentals such as SQL programming, database normalization, designing relational schema, and performing CRUD operations. It includes multiple constraints, joins, triggers, and structured table creation.	CO3, CO4, CO5	PO1, PO2
K5	Engineering Design	We designed a complete ER Diagram & Schema Diagram for tables such as Management, Customer, Venue, Event, Staff, Package, Payment, Review, and Event_History. All tables follow relationships, foreign keys, and business rules, representing engineering design processes.	CO3, CO5	PO2, PO3
K6	Engineering Practice	We implemented the database using MySQL Workbench. We created tables, inserted real-world sample data, applied updates, ran advanced SQL queries, and implemented a trigger to prevent double booking demonstrating practical engineering application.	CO3, CO4, CO5	PO1

P’s (Problem-solving Skills) Mapping

P’s	Attributes	How P’s are addressed through the project	CO	PO
P1	Depth of Knowledge Required	The system requires understanding DBMS concepts, schema design, SQL queries, constraints, triggers, Joins, and ERD modeling. It involves designing an optimized structure for Event Management operations.	CO3, CO4, CO5	PO1, PO2, PO3, PO7
P3	Depth of Analysis Required	Designing tables for events, payments, customers, venues, triggers, and packages needed analytical thinking. Abstract reasoning was required to avoid redundancy, maintain relationships, and implement real-world booking logic.	CO3, CO5	PO3
P7	Interdependency	All tables in the database are interconnected: Events depend on customers, venues, packages, staff; reviews depend on customers and events; payments depend on events. This shows how system components rely on one another for proper functioning.	CO3, CO4, CO5	PO2, PO7

A's (Attributes) Mapping

A's	Attributes	How A's are addressed through our project	CO	PO
A1	Range of Resources	Various resources were needed including MySQL Workbench, developer tools, sample business data, payment system logic, venue and package information, and event management workflow.	CO1	PO5
A2	Familiarity	We gained familiarity with event management operations, business processes, customer handling, scheduling, financial tracking, payment processing, and administrative workflow.	CO8, CO9	PO11

9.Conclusion

This project successfully demonstrates the design and implementation of a **CEP (Complex Event Processing) Mapping System** capable of capturing, filtering, analyzing, and correlating real-time data streams with precision. By applying structured mapping rules and event-driven logic, the system transforms raw incoming data into meaningful insights while enabling rapid detection of complex event patterns.

The work completed in this project proves the effectiveness of CEP in modern data-driven environments, especially where immediate responsiveness and intelligent automation are essential. The system not only meets its intended objectives but also provides a scalable and adaptable foundation for future analytical and monitoring applications.

Advantages

1. **Real-Time Event Processing:**
   Immediate detection and response to incoming events allow faster decision-making.
2. **High Accuracy Through Structured Mapping:**
   CEP mapping reduces noise, eliminates redundant information, and ensures consistent interpretation of data.
3. **Scalable & Modular Design:**
   Components can be expanded independently, making the system adaptable for larger and more complex environments.
4. **Enhanced Insight Generation:**
   The system correlates multiple event streams, giving deeper contextual understanding than traditional single-source systems.
5. **Supports Automation:**
   Can trigger alerts, workflows, or responses automatically based on detected event patterns.

Limitations

1. **High Computational Requirements:**
   Real-time analysis of continuous streams can demand powerful hardware, especially at large scales.
2. **Complex Rule & Pattern Configuration:**
   Designing accurate event rules requires domain expertise and can be time-consuming.
3. **Dependence on Data Quality:**
   Incomplete, inconsistent, or faulty data may lead to incorrect event detection.
4. **Scalability Challenges at Extreme Loads:**
   Massive parallel event streams may introduce latency without proper infrastructure optimization.
5. **Limited Adaptability Without Manual Updates:**
   Changes in business logic or event patterns often require manual reconfiguration.

Future Improvements

1. **Machine Learning Integration:**
   Incorporate ML-based prediction models to automatically learn event patterns and improve detection accuracy.
2. **Dynamic Rule Adjustment:**
   Allow the system to update or optimize event rules automatically based on historical data and performance trends.
3. **Distributed Processing Framework:**
   Use technologies like Apache Kafka, Flink, or Spark to handle extremely large event volumes with lower latency.
4. **Advanced Visualization Dashboard:**
   Add graphical dashboards for better monitoring, visualization, and real-time analytics.
5. **Fault-Tolerant Architecture:**
   Improve system reliability through redundancy, automatic recovery, and backup mechanisms.
6. **Enhanced Security Features:**
   Add encryption, access control, and anomaly detection to protect event data from misuse.